Angular Learning Log

Prompts, responses, implementation notes, and verified outcomes | Updated 2 September 2026

1. Fixing the Angular CLI PowerShell error

USER PROMPT

What is the issue and fix this

The attached screenshot showed:

```
PS C:\Projects\...\kiran-first-angular-app> ng build
ng : File C:\Users\kiran\AppData\Roaming\npm\ng.ps1 cannot be loaded because running scripts is
disabled on this system.
CategoryInfo          : SecurityError: (:) [], PSSecurityException
FullyQualifiedErrorId : UnauthorizedAccess
```

ASSISTANT RESPONSE

The Angular project was not the problem. PowerShell was blocking the `ng.ps1` launcher because the effective execution policy was `Restricted`.

The current-user policy was changed with:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

This scopes the change to the current Windows account and continues to block unsigned scripts downloaded from the internet.

`ng build` then completed successfully and generated `dist\kiran-first-angular-app`.

2. Creating the learning document

USER PROMPT

Add all the prompts and responses to be saved to learnings.pdf

ASSISTANT RESPONSE

Created and validated `learnings.pdf` in the project root with the prompts, error transcription, diagnosis, applied fix, and build verification.

3. Reviewing the Angular 7 backup

USER PROMPT

`Backup-02-After-Creating-Welcome-Component.zip`

What are the features included in this zip file?

ASSISTANT RESPONSE

The ZIP is an early Angular 7 Todo application snapshot containing:

- A root component showing `Welcome to todo!`, `Welcome to in28Minutes`, an Angular logo, and placeholder component content.
- A registered `WelcomeComponent` rendered through `<app-welcome>`, displaying `welcome works!`.
- Routing infrastructure and a router outlet, but no configured routes.
- Development and production build configurations.
- Karma/Jasmine unit tests, Protractor E2E setup, TSLint, and scripts for serving, building, testing, linting, and E2E testing.

It does not yet contain Todo CRUD, authentication, API integration, persistence, forms, navigation pages, or meaningful application styling. Some generated tests also appear stale after the welcome additions.

4. Adding comparable features to Angular 21

USER PROMPT

Add similar features in the current application

ASSISTANT RESPONSE

The relevant backup features were implemented using current Angular 21 conventions:

- Replaced the default Angular starter screen with a responsive welcome page.
- Added the `todo` title and `Welcome to in28Minutes` message.
- Created and embedded a standalone `Welcome` component.
- Added an Angular-style visual mark and component-content section.
- Retained the router outlet and existing empty route configuration.
- Added component tests and updated the root application tests.

Verification: 5 tests passed across 2 test files, and the production build completed successfully.

An unrelated OneNote metadata file changed externally during the work and was intentionally left untouched.

5. Ongoing updates

USER PROMPT

Keep updating the learnings.pdf

ASSISTANT RESPONSE

`learnings.pdf` was updated with the ZIP review, Angular 21 implementation, tests, build results, and this request. A maintainable HTML transcript source was retained so later conversation updates can be appended consistently.

6. Running the application

USER PROMPT

run the project

ASSISTANT RESPONSE

The Angular development server was started with:

```
npm start -- --host 127.0.0.1
```

The application responded with HTTP 200 and is available at `http://127.0.0.1:4200/`.

7. Interactive Angular Architecture Lab

USER PROMPT

`angular-architecture (1).jpg`

Implement the features shown in the image in this application, in demoable fashion, record all the ng commands used along with files produced. Document each feature that is implemented in the pdf file. Keep updating single file do not generate multiple pdfs.

ASSISTANT RESPONSE

Implemented a single interactive Angular Architecture Lab covering all eight concepts from the reference image. The existing development server hot-reloaded the feature at `http://127.0.0.1:4200/`.

Verification: 13 tests passed in 6 test files, the production build succeeded, and a headless-browser DOM check confirmed the rendered lab, binding controls, welcome component, and all 8 architecture cards.

Implemented features

Dependency Injection

`ArchitectureLab` obtains `ArchitectureLearning` with `inject()`. It also injects the typed `LAB_TITLE` token, which is configured by the feature module. The title and shared learning state visibly come from injected dependencies.

Modules

`DemoShellModule` imports and exports the standalone architecture lab and provides the title token. The standalone root component consumes that NgModule, demonstrating how module-packaged and standalone APIs interoperate in Angular 21.

Components

The page is split into reusable `ArchitectureLab`, `ConceptCard`, and existing `Welcome` components. Inputs send concept/selection/completion state into each card, while outputs send user actions back to the parent.

Templates

External HTML templates define the views. Angular's `@for` block renders the eight cards and file lists; `@if` renders the selected concept detail panel.

Metadata

The implementation uses `@Component`, `@Directive`, `@Injectable`, and `@NgModule` metadata. Selecting each concept displays a representative metadata/code example and its participating files.

Data Binding

The live binding panel demonstrates interpolation for the greeting and counter, property binding for disabled/progress/style values and child inputs, event binding for buttons and child outputs, and two-way `[(ngModel)]` binding for the learner name.

Directives

The standalone `appHighlight` attribute directive sets a CSS custom property and applies an active host class. Selecting a concept visibly highlights and raises that card. Built-in binding and control-flow directives are also exercised throughout the templates.

Services

The root-provided `ArchitectureLearning` service owns the eight-item catalog, completed-item set, computed count, and computed progress percentage. Marking cards learned and resetting the lab prove that shared state and logic are separated from presentation.

Angular CLI command ledger

Command used	Outcome and files produced
`ng generate component architecture-demo --standalone --skip-tests=false --style=css`	Stopped by an existing OneNote metadata-file merge conflict; no Angular component source was produced.
`ng generate component architecture-demo/concept-card --standalone --skip-tests=false --style=css`	`concept-card.ts`, `concept-card.html`, `concept-card.css`, `concept-card.spec.ts`
`ng generate service architecture-demo/services/architecture-learning --skip-tests=false`	`architecture-learning.ts`, `architecture-learning.spec.ts`
`ng generate directive architecture-demo/directives/highlight --standalone --skip-tests=false`	`highlight.ts`, `highlight.spec.ts`
`ng generate module architecture-demo/architecture-demo`	Stopped by the OneNote metadata-file merge conflict; no Angular module source was produced.
`ng generate interface architecture-demo/models/architecture-concept`	`architecture-concept.ts`
`ng generate component architecture-demo/architecture-lab --standalone --skip-tests=false --style=css`	Run four times: three attempts exposed parent/template OneNote conflicts; after isolating those generated conflict files, it produced `architecture-lab.ts`, `.html`, `.css`, and `.spec.ts`.
`ng generate component architecture-demo/architecture-lab --standalone --skip-tests=false --style=css --force`	One retry; still stopped because the conflict was a binary OneNote metadata file rather than an overwriteable Angular source file.
`ng generate module architecture-demo/demo-shell`	Run twice: the initial attempt hit the OneNote conflict; the successful retry produced `demo-shell/demo-shell-module.ts`.
`ng generate module architecture-demo/demo-shell --force`	One retry; it still stopped at the binary metadata conflict.
`ng test --watch=false`	The first run found an invalid direct directive-constructor test. After correcting the test to use an Angular host component, the second run passed all 13 tests.
`ng build`	Produced the optimized application in `dist/kiran-first-angular-app`; main bundle raw size was 246.93 kB.
`ng serve --host 127.0.0.1`	The existing development server rebuilt the changed files and served the demo at `http://127.0.0.1:4200/`.

Implementation file map

File	Purpose
`architecture-lab/architecture-lab.ts|html|css`	Interactive parent feature, external template, and responsive visual design.
`architecture-lab/architecture-lab.spec.ts`	Lab creation, eight-card rendering, and event-binding tests.
`concept-card/concept-card.ts|html|css`	Reusable input/output card component.
`concept-card/concept-card.spec.ts`	Required-input and rendered-metadata tests.
`services/architecture-learning.ts|spec.ts`	Injectable catalog/progress service and state tests.
`directives/highlight.ts|spec.ts`	Custom host-styling directive and Angular-host test.
`models/architecture-concept.ts`	Typed concept ID union and concept interface.
`demo-shell/demo-shell-module.ts`	NgModule packaging, export, and token provider.
`tokens/lab-title.ts`	Typed dependency-injection token; added during implementation.
`app.ts|html|css|spec.ts`	Root integration and architecture-lab rendering test.

Environment note: OneNote's `Open Notebook.onetoc2` metadata exists inside multiple project and schematic directories. Angular CLI attempted to copy these unrelated binary files while generating code, causing the recorded merge conflicts and incidental metadata files. They are not Angular features and are not imported by the application.

8. Removing OneNote metadata

USER PROMPT

delete all the onenote files in the application.

ASSISTANT RESPONSE

Searched the complete project tree for files with `.one`, `.onetoc2`, or `.onetmp` extensions and files named `Open Notebook*`. Every resolved target was verified to be inside the application workspace before deletion.

Deleted 2,356 OneNote-related files totaling 12,974,704 bytes. A second recursive scan found 0 remaining OneNote files.

The deletion is permanent rather than a recycle-bin operation. Most deleted files were metadata copies under `node_modules`; Angular application source files were not targeted.